Основы анализа данных

HMDA и табличные вычисления

Лекция
Перерыв
Практика

Реальные заявки и смысл полей

HMDA: публичные ипотечные записи США
Шесть строк для разбора; суммы в USD

source_row	action_taken	loan_amount, USD	income, тыс. USD/год
2	1	275 000	72
3	1	585 000	174
278	2	475 000	108
304	2	905 000	353
21	3	595 000	184
27	3	1 805 000	667

Какие данные будем читать

2023 год · Вашингтон · покупка жилья
Основной расчёт: выдано, одобрено, отказано

Файл	Строк	Роль
reading_example.csv	6	Разобрать команды вручную
decisions.csv	6 830	Посчитать доли для исходов 1–3
hmda_2023_dc_raw.csv	8 575	Исходный снимок, коды 1–5

Четыре типа аналитических вопросов

Одна таблица позволяет задать разные вопросы
Сегодня считаем, что наблюдалось

Тип вопроса	Пример
Описание	Какую долю заявок одобрили?
Диагностика	С чем связано различие организаций?
Прогноз	Одобрят ли новую заявку?
Причинный	Что изменит упрощение анкеты?

Зачем нужна pandas

pandas — библиотека для работы с таблицами

DataFrame — таблица со строками и столбцами

Series — один ряд значений с метками

pd, функция, метод и атрибут

import pandas as pd — загрузить библиотеку

pd.read_csv(...) — функция; table.head(...) — метод

table.shape — атрибут с размером таблицы

```
from pathlib import Path
import pandas as pd
DATA = Path("../data")
assert DATA.is_dir()
```

Демонстрация: B06 в lecture_demo.ipynb.

CSV и первое чтение read_csv

CSV: заголовок и значения, разделённые запятыми
example = pd.read_csv(DATA / "reading_example.csv")

Поле	Что хранится
source_row	Номер строки исходного CSV
activity_year	Отчётный год: 2023
lei	Код финансовой организации
county_code	Код территории: 11001
action_taken	Исход: 1, 2 или 3
loan_amount	Сумма, USD
income	Доход, тысячи USD в год

Демонстрация: B07 в lecture_demo.ipynb.

Параметры read_csv

sep — чем разделены поля файла

dtype — как хранить значения столбцов

nrows — сколько строк; usecols — какие столбцы

Неверный sep и диагностика

В CSV стоят запятые, а sep задан как "|"

Результат: шесть строк и один столбец

Проверяем таблицу после чтения

```
wrong_sep = pd.read_csv(DATA / "reading_example.csv", sep="|")
wrong_sep
```

Демонстрация: B09 в lecture_demo.ipynb.

dtype и сохранение идентификаторов

lei и county_code — коды, читаем их как строки

dtype={"lei": "string", "county_code": "string"}

action_taken пока остаётся целым числом

```
example = pd.read_csv(
    DATA / "reading_example.csv",
    dtype={"lei": "string", "county_code": "string"},
)
```

Демонстрация: B10 в lecture_demo.ipynb.

nrows и usecols

nrows=3 — загрузить первые три записи

usecols=["action_taken", "income"] — два столбца

Файлы на диске остаются прежними

```
preview = pd.read_csv(
    DATA / "reading_example.csv", nrows=3
)
preview
```

Демонстрация: B11 в lecture_demo.ipynb.

Размер, имена и типы столбцов

shape — количество строк и столбцов

columns — имена; dtypes — типы столбцов

example: (6, 7); wrong_sep: (6, 1)

example.shape
example.columns
example.dtypes

Демонстрация: B12 в lecture_demo.ipynb.

Выбор одного столбца и Series

outcomes = example["action_taken"]

Выбрали один столбец из таблицы example

Series: коды 1, 1, 2, 2, 3, 3

```
outcomes = example["action_taken"]
print(type(outcomes))
outcomes
```

Демонстрация: B13 в lecture_demo.ipynb.

Список столбцов и двойные скобки

names = ["action_taken", "income", "loan_amount"]

example[names] — таблица с тремя столбцами

example[["action_taken"]] — таблица с одним

```
names = ["action_taken", "income", "loan_amount"]
selected = example[names]
print(type(selected))
selected
```

Демонстрация: B14 в lecture_demo.ipynb.

head и цепочка выбора

head(3) показывает первые три строки таблицы

Выбор двух полей и просмотр первых шести строк

nrows — при чтении; head — после чтения

```
example.head(3)
example[["action_taken", "income"]].head(6)
```

Демонстрация: B15 в lecture_demo.ipynb.

info, isna и неизвестные значения

info() показывает заполненность и типы

isna() отмечает пропуски значением True

Пропуск означает отсутствие значения

```
example.info()
example["income"].isna()
```

Демонстрация: B16 в lecture_demo.ipynb.

value_counts и частоты категорий

value_counts() считает каждое значение

Для примера: по две записи с кодами 1, 2 и 3

normalize=True возвращает доли, а не проценты

outcomes.value_counts(dropna=False)

Демонстрация: B17 в lecture_demo.ipynb.

Метрики и ручной расчёт

Одобрения: 4 из 6; выдачи: 2 из 6
Общее количество — знаменатель: шесть записей

Код	Одобрение?	Выдача?
1	True	True
1	True	True
2	True	False
2	True	False
3	False	False
3	False	False

eq и isin: условия для каждой строки

eq(1) — равен ли исход единице?
isin([1, 2]) — входит ли исход в список?

Код	eq(1)	isin([1, 2])
1	True	True
2	False	True
3	False	False

Демонстрация: B19 в lecture_demo.ipynb.

sum, len и проверяемая доля

sum() считает True; len(example) даёт шесть

Одобрения: 4 / 6; выдачи: 2 / 6

assert останавливает выполнение при False

```
n_approved = approved_flag.sum()
n_decisions = len(example)
assert n_decisions > 0
approval_rate = n_approved / n_decisions
```

Демонстрация: B20 в lecture_demo.ipynb.

Полный срез: 6 830 решений

decisions.csv: 6 830 записей с исходами 1–3

Одобрения: 6 139 / 6 830 = 89,88%

Выдачи: 5 956 / 6 830 = 87,20%

```
full_outcomes = decisions["action_taken"]
full_approved = full_outcomes.isin([1, 2]).sum()
full_issued = full_outcomes.eq(1).sum()
full_n = len(decisions)
```

Демонстрация: B21 в lecture_demo.ipynb.

Ошибка определения и ошибка типа

eq(1) считает выдачи; isin([1, 2]) — одобрения

Число 1 и строка "1" сравниваются по-разному

Проверяем определение показателя и тип данных

```
text_copy = pd.read_csv(
    DATA / "reading_example.csv",
    dtype={"action_taken": "string"},
)
text_copy["action_taken"].eq(1).sum()
text_copy["action_taken"].eq("1").sum()
```

Демонстрация: B22 в lecture_demo.ipynb.

Индекс: метка и позиция

index — метки строк; позиция — место в порядке

source_row уже есть в нашей таблице

indexed = example.set_index("source_row")

example.index

Демонстрация: B23 в lecture_demo.ipynb.

loc и iloc

loc[278, "action_taken"] — по меткам

iloc[2, 3] — по позициям, начиная с нуля

В indexed оба обращения возвращают код 2

Демонстрация: B24 в lecture_demo.ipynb.

Фильтрация строк

denied_mask = decisions["action_taken"].eq(3)

True отмечает строки с отказом

loc[условие, список полей] выбирает записи

```
denied_mask = decisions["action_taken"].eq(3)
names = ["source_row", "lei", "loan_amount", "income"]
denied = decisions.loc[denied_mask, names]
denied.head(5)
```

Демонстрация: B25 в lecture_demo.ipynb.

Несколько условий

gt(1_000_000) — сумма больше миллиона

& — оба условия; | — хотя бы одно; ~ — отрицание

1_000_000 — обычное число Python

```
large_mask = decisions["loan_amount"].gt(1_000_000)
large_denied_mask = denied_mask & large_mask
names = ["source_row", "loan_amount", "action_taken"]
large_denied = decisions.loc[large_denied_mask, names]
large_denied.head(5)
```

Демонстрация: B26 в lecture_demo.ipynb.

Сортировка и пять крупнейших сумм

sort_values сортирует строки по значениям

ascending=False — от большего к меньшему

Сначала сортировка, затем head(5)

```
denied_sorted = denied.sort_values(
    ["loan_amount", "source_row"],
    ascending=[False, True],
)
denied_sorted.head(5)
```

Демонстрация: B27 в lecture_demo.ipynb.

Вычисляемый столбец и единицы

income хранится в тысячах USD в год

income_usd = income × 1000

Новый столбец добавляем в копию prepared

```
prepared = decisions.copy()
prepared["income_usd"] = prepared["income"] * 1000
prepared[["income", "income_usd"]].head(5)
```

Демонстрация: B28 в lecture_demo.ipynb.

NumPy и векторные вычисления

NumPy хранит числовые массивы

to_numpy() берёт значения Series без меток

Умножение на 1000 применяется к каждому числу

```
income_array = decisions["income"].to_numpy()
print(type(income_array))
print(income_array.shape, income_array.dtype)
income_array[:6]
```

Демонстрация: B29 в lecture_demo.ipynb.

groupby и новая единица наблюдения

lei — код организации, повторяется в заявках

groupby("lei") собирает записи по организациям

size() — число записей; mean() — средняя сумма

```
groups = decisions.groupby("lei", dropna=False)
organization_sizes = groups.size()
organization_mean_amount = groups["loan_amount"].mean()
```

Демонстрация: B30 в lecture_demo.ipynb.

agg и профиль организаций

is_approved: True для исходов 1 и 2

agg собирает количество, одобрения и среднюю сумму

Долю каждой организации считаем по её записям

```
prepared["is_approved"] = prepared["action_taken"].isin([1, 2])
profile = prepared.groupby("lei", dropna=False).agg(
    n_decisions=("action_taken", "size"),
    n_approved=("is_approved", "sum"),
    mean_loan_usd=("loan_amount", "mean"),
).reset_index()
```

Демонстрация: B31 в lecture_demo.ipynb.

Интерпретация и передача результата

89,88% одобрений; 87,20% выдач

Вашингтон · 2023 · покупка жилья · исходы 1–3

Сохраняем код, определения и порядок запуска

Закрепление и переход к практике

Прочитать и проверить CSV

Посчитать одобрения и выдачи

Заполнить README и повторить чистый запуск